

		<pre> 63 Dictionary dictArray[MAX]; 64 int count = 0; </pre>	(Array). Menampilkan seluruh kata di dalam memori/array secara berurutan sesuai input awal pengguna.
2	DCLL	<pre> 25 typedef struct Node { //DCLL 26 Dictionary data; 27 struct Node* next; 28 struct Node* prev; 29 } Node; 118 void insertDCLL(Dictionary d) { //sisipkan di akhir DCLL 119 Node* newNode = (Node*)malloc(sizeof(Node)); 120 newNode->data = d; 121 122 if(head == NULL) { 123 head = newNode; 124 newNode->next = newNode; 125 newNode->prev = newNode; 126 } else { 127 Node* tail = head->prev; 128 tail->next = newNode; 129 newNode->prev = tail; 130 newNode->next = head; 131 head->prev = newNode; 132 } 133 } 135 void deleteDCLL(char word[]) { 136 if(head == NULL) return; 137 Node* curr = head; 138 do { 139 if(strcmp(curr->data.indo, word) == 0) { 140 if(curr->next == curr) { 141 free(curr); 142 head = NULL; 143 return; 144 } 145 curr->prev->next = curr->next; 146 curr->next->prev = curr->prev; 147 if(curr == head) head = curr->next; 148 free(curr); 149 return; 150 } 151 curr = curr->next; 152 } while(curr != head); 153 } </pre>	Halaman 2, 4, 5 - 6 Deskripsi: <i>Struktur Latar Belakang</i> untuk Operasi CRUD: Menu 1 (Add Word), Menu 2 (Delete Word), dan Menu 3 (Change Translate Word).
3	BST	<pre> 31 typedef struct TreeNode { //BST 32 Dictionary data; 33 struct TreeNode* left; 34 struct TreeNode* right; 35 } TreeNode; 157 int bstCmp(const char *a, const char *b) { //perbandingan case-insensitive untuk BST 158 char la[100], lb[100]; 159 toLowerStr(a, la, sizeof(la)); 160 toLowerStr(b, lb, sizeof(lb)); 161 return strcmp(la, lb); 162 } </pre>	Halaman 11 - 12 Deskripsi: Menu 6: <i>Display Sorted (BST)</i>. Menampilkan kata yang

		<pre> 164 TreeNode* minValueNode(TreeNode* node) { //cari node terkecil (paling kiri) 165 TreeNode* curr = node; 166 while(curr && curr->left != NULL) curr = curr->left; 167 return curr; 168 } 170 TreeNode* insertBST(TreeNode* node, Dictionary d) { //sisipkan ke BST (urut A-Z, case-insensitive) 171 if(node == NULL) { 172 TreeNode* newNode = (TreeNode*)malloc(sizeof(TreeNode)); 173 newNode->data = d; 174 newNode->left = NULL; 175 newNode->right = NULL; 176 return newNode; 177 } 178 int cmp = bstCmp(d.indo, node->data.indo); 179 if (cmp < 0) node->left = insertBST(node->left, d); 180 else if (cmp > 0) node->right = insertBST(node->right, d); 181 return node; 182 } 184 TreeNode* deleteBST(TreeNode* node, char word[]) { 185 if (!node) return NULL; 186 int cmp = bstCmp(word, node->data.indo); 187 188 if (cmp < 0) node->left = deleteBST(node->left, word); 189 else if (cmp > 0) node->right = deleteBST(node->right, word); 190 else { 191 if (!node->left) { 192 TreeNode* tmp = node->right; 193 free(node); 194 return tmp; 195 } 196 if (!node->right) { 197 TreeNode* tmp = node->left; 198 free(node); 199 return tmp; 200 } 201 TreeNode* tmp = minValueNode(node->right); 202 203 node->data = tmp->data; 204 node->right = deleteBST(node->right, tmp->data.indo); 205 } 206 return node; 207 } 209 void inorder(TreeNode* node) { //tampilkan BST secara in-order (terurut A-Z) 210 if(node != NULL) { 211 inorder(node->left); 212 printf(" %-22s -> %s\n", node->data.indo, node->data.eng); 213 inorder(node->right); 214 } 215 } </pre>	sudah tersusun rapi secara alfabetis (A-Z) menggunakan teknik penelusuran <i>In-Order traversal</i>.
4	Stack	<pre> 42 typedef struct stackNode { //Stack 43 int operation; 44 Dictionary oldData; 45 Dictionary newData; 46 struct stackNode* next; 47 } stackNode; 74 stackNode* top = NULL; 266 /* Stack */ 267 void pushStack(int operator, Dictionary oldData, Dictionary newData) { //push operasi ke stack 268 stackNode* newNode = (stackNode*)malloc(sizeof(stackNode)); 269 270 newNod->operation = operator; 271 newNod->oldData = oldData; 272 newNod->newData = newData; 273 newNod->next = top; 274 top = newNod; 275 } 276 int popStack(int* operator, Dictionary* oldData, Dictionary* newData) { //pop operasi dari stack; return 0 jika kosong 277 if(top == NULL) return 0; 278 stackNode* temp = top; 279 280 *operator = temp->operation; 281 *oldData = temp->oldData; 282 *newData = temp->newData; 283 temp = temp->next; 284 free(temp); 285 return 1; 286 } 287 int isEmptyStack(void) { //cek apa stack kosong 288 return (top == NULL); 289 } </pre>	Halaman 13 - 14 Deskripsi: Menu 7: <i>Undo Last Operation (Stack)</i>. Digunakan untuk membatalkan aksi modifikasi terakhir dengan prinsip LIFO (<i>Last In, First Out</i>).

5	Queue	<pre> 48 49 struct QueueNode { //Queue 50 char keyword[100]; 51 char translation[100]; 52 struct QueueNode* next; 53 } QueueNode; 54 76 QueueNode* front = NULL; 77 QueueNode* rear = NULL; </pre> <pre> 294 /* ===== QUEUE ===== */ 295 void enqueueHistory(char *keyword, char *translation) { //enqueue kata yang dicari; otomatis hapus jika penuh 296 //hapus elemen terlama jika antrian sudah penuh 297 if (qSize >= MAX_HISTORY) { 298 QueueNode *tmp = front; 299 front = front->next; 300 if (!front) rear = NULL; 301 free(tmp); 302 qSize--; 303 } 304 QueueNode *newNode = (QueueNode *) malloc(sizeof(QueueNode)); 305 strncpy(newNode->keyword, keyword, 99); 306 newNode->keyword[99] = '\0'; 307 strncpy(newNode->translation, translation, 99); 308 newNode->translation[99] = '\0'; 309 newNode->next = NULL; 310 311 if (!rear) front = rear = newNode; 312 else { 313 rear->next = newNode; 314 rear = newNode; 315 } 316 qSize++; 317 } 318 319 void displayHistory(void) { 320 if (!front) { 321 printf(" [Still no search history exist]\n"); 322 return; 323 } 324 printf("\n Search History (oldest -> newest):\n"); 325 printf(" %-5s %-25s %s\n", "No.", "Searched Word", "Translation"); 326 printf(" %-5s %-25s %s\n", "---", "-----", "-----"); 327 int i = 1; 328 QueueNode *cur = front; 329 while (cur) { 330 printf(" %-5d %-25s %s\n", i++, cur->keyword, cur->translation); 331 cur = cur->next; 332 } 333 printf("\n Total: %d history (maks. %d)\n", qSize, MAX_HISTORY); 334 } 335 336 void searchHistory(void) { 337 displayHistory(); 338 } </pre>	Halaman 15 - 16 Deskripsi: Menu 8: <i>Word Search History</i>. Menampilkan riwayat pencarian kata terakhir Anda secara berurutan menggunakan prinsip FIFO (<i>First In, First Out</i>).
6	Hash	struct: <pre> 37 typedef struct HashNode { //Hash 38 Dictionary data; 39 struct HashNode* next; 40 } HashNode; </pre> function: <pre> 218 /* ===== HASH ===== */ 219 int hashFunc(const char *word) { 220 unsigned int sum = 0; 221 for (int i = 0; word[i] != '\0'; i++) sum += (unsigned char) tolower((unsigned char)word[i]); 222 return (int) (sum % HASH_SIZE); 223 } 224 225 void insertHash(Dictionary d) { 226 int index = hashFunc(d Indo); 227 HashNode* newNode = (HashNode*) malloc(sizeof(HashNode)); 228 229 newNode->data = d; 230 newNode->next = hashTable[index]; 231 hashTable[index] = newNode; 232 } 233 234 HashNode* searchHash(char word[]) { 235 int index = hashFunc(word); 236 HashNode* temp = hashTable[index]; 237 238 while (temp != NULL) { 239 if (bstCmp(temp->data Indo, word) == 0) return temp; 240 temp = temp->next; 241 } 242 return NULL; 243 } 244 245 void deleteHash(char word[]) { 246 int index = hashFunc(word); 247 HashNode* curr = hashTable[index]; 248 HashNode* prev = NULL; 249 250 while (curr != NULL) { 251 if (bstCmp(curr->data Indo, word) == 0) { 252 if (prev == NULL) hashTable[index] = curr->next; 253 else prev->next = curr->next; 254 free(curr); 255 return; 256 } 257 prev = curr; 258 curr = curr->next; 259 } 260 } 261 262 void updateHash(char word[], char newTranslation[]) { 263 HashNode* node = searchHash(word); 264 if (node != NULL) strcpy(node->data Eng, newTranslation); 265 } </pre>	Halaman 7 - 9 Deskripsi: Menu 4: <i>Search Word (Hash + Trie)</i>. Digunakan saat ingin mencari kata secara langsung (<i>direct hit</i>) tanpa penelusuran satu per satu.
7	Trie	struct: <pre> 55 typedef struct TrieNode { //Trie 56 struct TrieNode* child[TRIE_ALPHA]; 57 int endWord; 58 char fullWord[100]; 59 } TrieNode; </pre> function:	Halaman 7 - 9, 17 - 18 Deskripsi: Menu 4: <i>Search Word</i> dan Menu 9: <i>Prefix Search</i>. Memfasilitasi pencarian

		<pre> 341 /* ===== TRIE ===== */ 342 TrieNode* createTrieNode() { 343 TrieNode* newNode = (TrieNode*) malloc(sizeof(TrieNode)); 344 newNode->endWord = 0; 345 newNode->fullWord[0] = '\0'; 346 for(int i = 0; i < TRIE_ALPHA; i++) newNode->child[i] = NULL; 347 return newNode; 348 } 349 350 void insertTrie(const char *word) { 351 if (!rootTrie) rootTrie = createTrieNode(); 352 TrieNode *cur = rootTrie; 353 354 for (int i = 0; word[i]; i++) { 355 int idx = trieIndex(word[i]); 356 357 if (idx < 0) continue; 358 if (!cur->child[idx]) cur->child[idx] = createTrieNode(); 359 cur = cur->child[idx]; 360 } 361 cur->endWord = 1; 362 strcpy(cur->fullWord, word, 99); 363 cur->fullWord[99] = '\0'; 364 } 365 366 int deleteTrieHelper(TrieNode* curr, char word[], int depth) { 367 if (curr == NULL) return 0; 368 if (word[depth] == '\0') { 369 curr->endWord = 0; 370 return 1; 371 } 372 int index = tolower(word[depth]) - 'a'; 373 374 if (index < 0 index > 25) return 0; 375 return deleteTrieHelper(curr->child[index], word, depth + 1); 376 } 377 378 void deleteTrie(char word[]) { 379 deleteTrieHelper(rootTrie, word, 0); 380 } 381 382 TrieNode* searchPrefix(const char *prefix) { //cari node akhir dari prefix (case-insensitive) 383 if (!rootTrie) return NULL; 384 TrieNode *cur = rootTrie; 385 for (int i = 0; prefix[i]; i++) { 386 int idx = trieIndex(tolower((unsigned char)prefix[i])); 387 if (idx < 0 !cur->child[idx]) return NULL; 388 cur = cur->child[idx]; 389 } 390 return cur; 391 } 392 393 void collectWords(TrieNode* node) { 394 if (node->endWord && suggestCount < MAX_SUGGEST) 395 strcpy(suggestList[suggestCount++], node->fullWord); 396 for (int i = 0; i < TRIE_ALPHA; i++) 397 if (node->child[i]) collectWords(node->child[i]); 398 } 399 400 </pre>	berbasis awalan (<i>prefix</i>) untuk menghasilkan fitur <i>auto-complete</i> atau memberikan rekomendasi kata
8	Heap	struct: <pre> 84 char heap[100][100]; 85 int heapSize = 0; 86 int loadedWords = 0; </pre> function: <pre> 401 /* ===== HEAP ===== */ 402 void insertHeap(char word[]) { //insert suggestion 403 if (heapSize == 10) return; 404 strcpy(heap[heapSize], word); 405 heapSize++; 406 } 407 408 void traversalTrie(TrieNode* curr, char word[], int level) { 409 if (curr->endWord) { 410 word[level] = '\0'; 411 insertHeap(word); 412 } 413 414 for (int i = 0; i < 26; i++) { 415 if (curr->child[i] != NULL) { 416 word[level] = i + 'a'; 417 traversalTrie(curr->child[i], word, level + 1); 418 } 419 } 420 } 421 422 void prefixSearch() { 423 char prefix[100]; 424 printf("Input Prefix: "); 425 scanf("%s", prefix); 426 clearInputBuffer(); 427 428 TrieNode* curr = rootTrie; 429 for (int i = 0; prefix[i] != '\0'; i++) { 430 431 int index = tolower((unsigned char)prefix[i]) - 'a'; 432 if (index < 0 index > 25) { 433 printf("Invalid prefix.\n"); 434 return; 435 } 436 437 if (curr->child[index] == NULL) { 438 printf("No matching words.\n"); 439 return; 440 } 441 curr = curr->child[index]; 442 } 443 444 heapSize = 0; 445 char temp[100]; 446 strcpy(temp, prefix); 447 traversalTrie(curr, temp, strlen(prefix)); 448 449 printf("Suggestions:\n"); 450 for (int i = 0; i < heapSize; i++) printf("%s\n", heap[i]); 451 } </pre>	Halaman 17 - 18 Deskripsi: Menu 9: <i>Prefix Search</i>. Bekerja dengan insertHeap() untuk menyimpan sementara hasil <i>Depth-First Search</i> (DFS) dari Trie ke dalam array heap, dengan batasan maksimal 10 kata rekomendasi.

9	File Processing	<pre> 454 // ===== FILE ===== */ 455 void loadFileO{ //muat kata dari file dictionary.txt saat program mulai 456 FILE* fp = fopen(FILENAME,"r"); 457 if(fp == NULL) return; 458 Dictionary d; 459 loadedWords = 0; 460 461 while(462 fscanf(fp, "%99[^\n]\n", d.indo, d.eng) == 2){ 463 if(count == MAX){ 464 printf("Dictionary is full.\n"); 465 break; 466 } 467 dictArray[count++] = d; 468 469 insertDCLL(d); 470 root = insertBST(root,d); 471 insertHash(d); 472 insertTrie(d.indo); 473 loadedWords++; 474 } 475 fclose(fp); 476 } 477 478 void saveFileO{ //simpan semua kata ke file dictionary.txt 479 FILE* fp = fopen(FILENAME,"w"); 480 if(fp == NULL){ 481 printf("Cannot save file.\n"); 482 return; 483 } 484 for(int i = 0; i < count; i++) {printf(fp, "%s\n", dictArray[i].indo, dictArray[i].eng); 485 fclose(fp); 486 printf("Successfully saved %d words to '%s'.\n", count, FILENAME); 487 } 488 489 </pre>	Halaman 1, 19 - 20 Deskripsi: Menu 0: <i>Save & Exit</i> dan <i>Loading</i> awal program. Menangani pemuatan data kamus saat <i>compile</i> awal. Memastikan persistensi data dengan fungsi <i>saveFile()</i> agar modifikasi RAM tersimpan di <i>dictionary.txt</i>.
---	-----------------	---	---

Assessment 2: Show that the Application able to run well [30 %]

A project with rigorous documentation CAN opt NOT to submit a live demo.

Recording Links of the project demo:

 **Screenshots documentation**

You may put “NO DEMO” instead if no live demo is provided.